Mini Language Specification

A specification on how our language should work and be understood by the compiler.

1. Variables and assignment

An assignment gives a value to a variable. The first assignment to a name declares that variable; its type is inferred from that first value. Later assignments update the existing variable and must produce a value of the same type.

```
x = 10

name = "Ada"

ready = true
```

Variables do not need a separate declaration keyword.

Multiple assignment

An assignment may name several variables and provide the same number of values:

```
a, b, c = 1, 2, 3

a, b = b, a
```

We evaluate the expressions on the right first before updating anything. This makes the swap example exchange the values of ‘a’ and ‘b’ correctly. The number of targets and values must match. Each target must be a valid variable

2. Static types

The language has four core types:

Type	Meaning	Example
'int'	Whole number	'10', '-2'
'float'	Number with a fractional part	'3.5', '0.25'
'string'	Text	"hello"
'bool'	Truth value	'true', 'false'

Types are checked statically: the compiler determines expression types and reports incompatible operations or assignments before the program runs. A variable's type is inferred once, from the value of its initial assignment.

3. Output

Use 'log: expression' to display the value of an expression:

```
log: "Hello world!"

log: 2 + 3
```

4. Arithmetic operators

The arithmetic operators are:

Operator	Meaning	Operand types / result
`+`	Addition	`int` with `int` gives `int`; numeric operands involving `float` give `float`
`-`	Subtraction	Numeric operands; result follows the same rule as `+`
`*`	Multiplication	Numeric operands; result follows the same rule as `+`
`/`	Division	Numeric operands; result is `float`
`%`	Remainder	`int` operands; result is `int`
`^`	Exponentiation	Numeric operands; result is `float` if either operand is `float`, otherwise `int` for a non-negative integer exponent

5. Comparison operators

Comparisons produce a ‘bool’ and may be used in conditions:

Operator	Meaning
`==`	Equal to
`!=`	Not equal to
`<`	Less than
`<=`	Less than or equal to
`>`	Greater than
`>=`	Greater than or equal to

6. Logical operators

Logical operators combine Boolean expressions and return a ‘bool’:

Operator	Meaning
‘!’	NOT: reverses a Boolean
‘&&’	AND: true only when both operands are true
‘||’	OR: true when at least one operand is true

if (age >= 18 && has_id) {

log: "allowed"

}

Logical operators require ‘bool’ operands. ‘&&’ and ‘||’ short-circuit: the right operand is evaluated only when needed to determine the result.

7. Precedence

Parentheses override the rules below. Operators on a higher row bind more tightly than operators on a lower row. Operators in the same row associate as shown.

Precedence (highest first)	Operators
1	Parentheses ‘( )’
2	Unary ‘!’, unary ‘+’, unary ‘-‘
3	‘^’
4	‘*’, ‘/’, ‘%’
5	‘+’, ‘-‘
6	‘<’, ‘<=’, ‘>’, ‘>=’
7	‘==’, ‘!=’
8	‘&&’
9	‘||’

8. Conditions

An ‘if’ body runs when its parenthesized boolean condition is true. An optional ‘else’ body runs when it is false:

```
if (x < 20) {
x = x + 1
} else {
x = x - 1
}
```

To test another condition after the first fails, use ‘else if’:

```
if (x < 10) {
x = x + 2
} else if (x < 20) {
x = x + 1
} else {
x = x - 1
}
```

Separate 'if' statements are independent and each condition is checked.

9. Loops

A counted loop repeats its body with a loop variable taking values from the start through the end:

```
loop i from 2 to 10 step 2 {
log: i
}
```

The "step" clause is optional. When omitted, the step is "1":

```
loop i from 1 to 3 {
log: i
}
```

Loop rules

1. Evaluate the start, end, and explicit step expressions once, before the first iteration.
2. The loop variable starts at the start value.

3. The end is inclusive. With a positive step, continue while the loop variable is less than or equal to the end. A negative step counts downward. The loop continues while the loop variable is greater than or equal to the inclusive end value.
4. The default step is '1'. A step of '0' is invalid and causes a runtime error before the loop body runs.
5. After each completed body, add the step to the loop variable. If the initial value is already beyond the end in the step's direction, the body runs zero times.
6. The loop variable is scoped to the loop and its body. It cannot be used after the loop. A same-named variable outside the loop is unaffected.